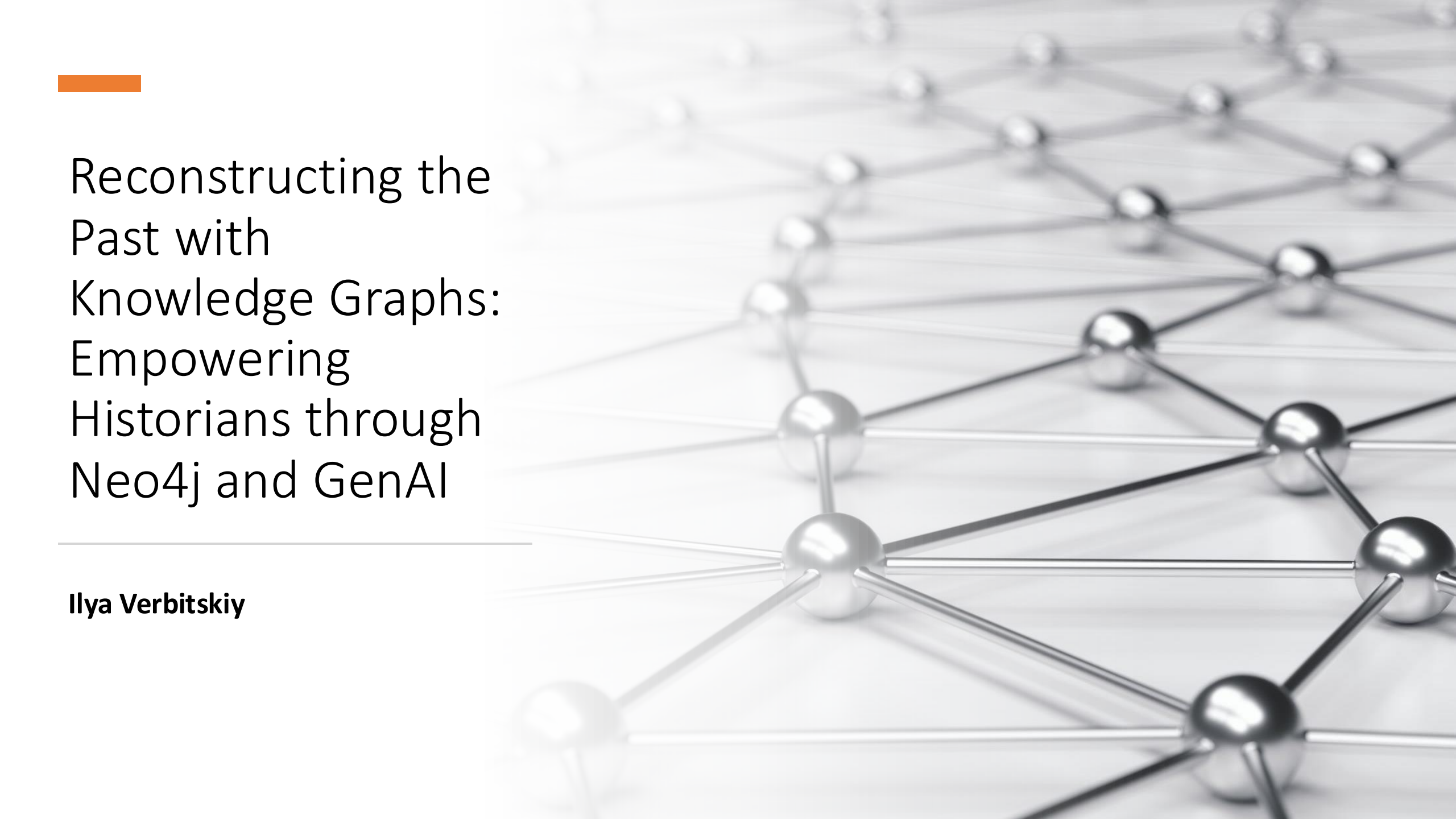
Reconstructing the Past with Knowledge Graphs: Empowering Historians through Neo4j and GenAI

Ilya Verbitskiy



Agenda

- Knowledge Graphs and Archaeology & Numismatics
- Understanding RAG (Retrieval-Augmented Generation)
- Enhancing RAG Relevance
- Agentic RAG
- Accessing Numismatics Data via MCP Server

Applications in Archaeology

- **Contextual linking** of excavation records, artifacts, sites, and historical periods.
- **Interoperability** between different datasets (museum collections, excavation logs, GIS systems).
- Supports **semantic search** and **cross-disciplinary research** (e.g., linking texts, maps, and artifact metadata).
- Example: Europeana, ARIADNE, or the British Museum's Linked Open Data projects, Nomisma.org

Application in Numismatics

- Connects **coins → rulers → mints → iconography → inscriptions → chronology**.
- Facilitates digital cataloguing and **comparison of coin types across collections.**
- Enables **provenance tracking** and authenticity verification.
- Supports data-driven insights into **trade routes, political influence, and economic history.**

Maiorina – Honorius (Obverse)

- Portrait right
- **Script:** Latin
- **Lettering:** DN HONORIVS PF AVG
- **Translation:** Our Lord Honorius, the Pious, the Blessed Emperor.



Maiorina – Honorius (Reverse)

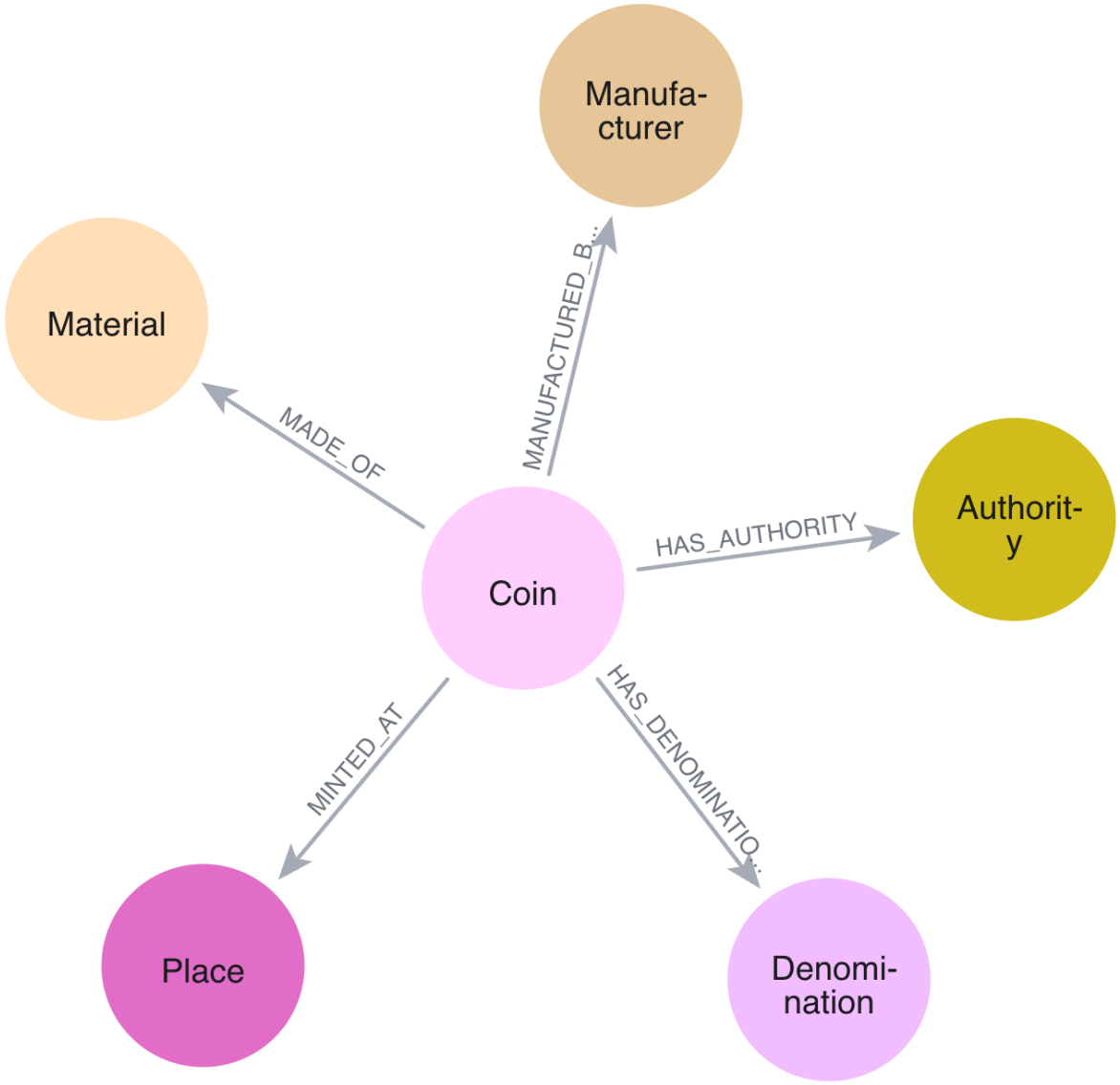
- Emperor standing frontally, dressed militarily, facing right, holding a chrismated labarum in his right hand and a globe in his left hand. Star in the right field.
- **Script:** Latin
- **Lettering:** GLORIA ROMANORVM SMHA
- **Translation:** The Glory of the Romans — struck at the Sacred Mint of Heraclea, first officina.



Online Coins of the Roman Empire

- OCRE is a **digital corpus/catalogue** of the imperial coinage of the Roman Empire (from the reign of Augustus in 31 BC to the death of Zeno in 491 AD).
- It is a **collaborative project** between the American Numismatic Society (ANS) and the Institute for the Study of the Ancient World (ISAW) at New York University (NYU).
- It uses linked open data, stable identifiers in the numismatic domain (via Nomisma.org), and is meant to be an **open resource** for both specialists and the broader public.

OCRE Database: Coin Relationship Graph



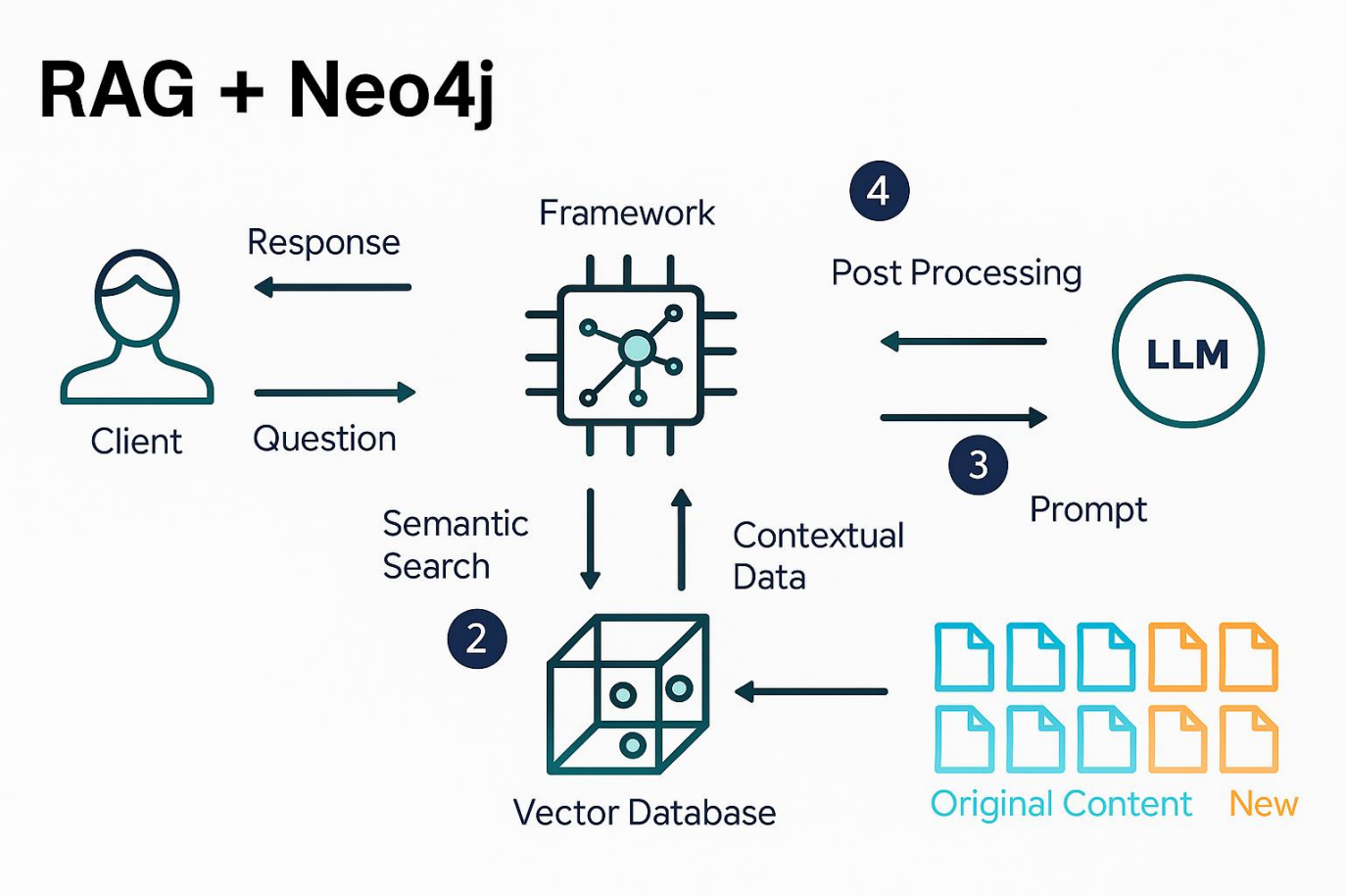
Core AI and Data Integration Tools

- Retrieval-Augmented Generation (RAG)
- Agentic RAG (autonomous data retrieval & orchestration)
- Model Context Protocol (MCP)
- Knowledge Graphs
- Linked Open Data (LOD)
- Vector Databases
- Ontology Management (OWL, RDF, SPARQL)
- Multimodal GenAI (text + image + spatial data)

What is RAG?

- Retrieval-Augmented Generation (RAG) is an AI technique that combines **information retrieval** with **text generation** to produce more accurate and grounded responses.
- Ensures **up-to-date and factual** responses
- Reduces **hallucination** by grounding outputs in real data
- Enables **domain-specific** knowledge integration

How does it work?



What is Vector Indexing?

- Vector indexing enables semantic search by storing and querying vector embeddings.
- Instead of matching exact keywords, it finds conceptually similar items based on proximity in embedding space.
- Useful for RAG-powered applications: recommendations, question answering, and document retrieval.
- **Embeddings** are numerical representations of text, images, or other data that capture meaning in a multi-dimensional space.
  - OpenAI text-embedding-3-small / 1536 dimensions
  - OpenAI text-embedding-3-large / 3072 dimensions
  - sentence-transformers/all-mpnet-base-v2 / 768 dimensions
- neo4j_graphrag.retrievers.VectorRetriever

How it works in Neo4j

```
CREATE VECTOR INDEX coins_embeddings_index IF NOT EXISTS
FOR (c:Coin)
ON (c.embedding)
OPTIONS { indexConfig: {
    'vector.dimensions': 1536,
    'vector.similarity_function': 'cosine'
}}
```

How it works in Neo4j

```
class CoinsIngestor:
    def __init__(self, dataset):
        self._openai = OpenAI(api_key="sk-...")
        # ...


    def _create_coins(self, record):
        description = record.get("description", "")
        embedding_response = self._openai.embeddings.create(
            model="text-embedding-3-small", input=description
        )
        embedding = embedding_response.data[0].embedding
        # ...
```

Full-Text Search Index

- A Full-Text Search Index allows efficient searching of text data stored in Neo4j.
- Supports fuzzy matching, relevance ranking, and multi-property search.
- Ideal for applications needing keyword-based or phrase-based lookup.

How it works in Neo4j

```
CREATE FULLTEXT INDEX coins_text_index IF NOT EXISTS
FOR (c:Coin)
ON EACH [c.title, c.description]
```

What is Hybrid Search?

- **Hybrid Search** combines **Full-Text Search** and **Vector Search** to achieve both **precision** and **semantic understanding**.
- It leverages keyword-based relevance from full-text search and contextual similarity from vector embeddings.
- neo4j_graphrag.retrievers. HybridRetriever

When Full-Text Search Works Better

- Ideal for **exact term matching** or **structured text queries**.
- Best used when users know specific keywords, phrases, or entity names.
- Suitable for scenarios like:
  - Searching document titles or metadata.
  - Filtering by specific attributes (e.g., author, tag, category)
  - Compliance or legal searches where exact wording matters.
- For example, “*What is the coin with DN HONORIVS PF AVG lettering?*”

When Vector Search Works Better

- Best for **semantic understanding** and **concept-based retrieval.**
- Useful when queries are descriptive or natural language-based.
- Ideal for scenarios like:
  - Finding conceptually similar documents.
  - Recommending related content even without shared keywords.
  - Semantic question answering.
  - For example, "*What are the Roman coins where an emperor standing frontally, dressed militarily, and facing right on the reverse?*"

What is GraphRAG?

- **GraphRAG** combines:
  - **Neo4j graph database** for structured knowledge.
  - **Vector search** for semantic similarity.
  - **LLM reasonin**g for natural language answers.
- It enables context-rich question answering over graph-connected data.

System Architecture

- **Neo4j** stores coin data as graph nodes and relationships.
- **Vector Index** stores embeddings for semantic search.
- **HybridRetriever** fetches relevant nodes using both:
  - Keyword/full-text search.
  - Vector similarity search.
- **OpenAI LLM** uses the retrieved context to generate the answer.
- **RagTemplate** defines how context and question are formatted for the model.

Code Setup

```
driver = GraphDatabase.driver(neo4j_uri, auth=(neo4j_username, neo4j_password))
embedder = OpenAIEmbeddings(model="text-embedding-3-small", api_key="sk-...")
retriever = HybridRetriever(driver, VECTOR_INDEX, FULL_TEXT_INDEX, embedder)
llm = OpenAILLM(
    "gpt-4o-mini",
    model_params={
        "temperature": 0.2,
        "top_p": 1.0,
    },
    api_key="sk-..."
)
prompt_template = RagTemplate(...)
graph_rag = GraphRAG(llm=llm, retriever=retriever, prompt_template=prompt_template)
```

Contextual Question Rewriting

What It Is:

A prompt engineering method that reformulates a user's latest question so it's self-contained — understandable without needing prior conversation context.

Key Idea:

Replace ambiguous references (like "he," "it," or "that report") with explicit details pulled from earlier dialogue.

This improves clarity, retrieval accuracy, and grounding for LLMs or RAG systems.

How It Works:

1. Detect implicit references or pronouns in the new user message
2. Look up their meaning in the recent conversation history
3. Substitute them explicitly in the rewritten version
4. Output a clear, standalone question

Step-back prompting

What It Is:

A prompting technique where the model is asked to step back and reflect on the broader context before answering, improving reasoning accuracy and reducing bias.

Key Idea:

Instead of jumping straight to the answer, the model first reasons at a higher level:

" What is the core problem here?" → then → "What is the solution?"

How It Works:

1. Ask for abstraction: Request a summary or general explanation first.

2. Guide reasoning: Encourage identifying relationships and principles.

3. Return to specifics: Apply those insights to the original question.

Example:

❌ "Who won the 2020 US election?"

✅ "Let’s think about how US presidential elections work, then identify the 2020 winner."

Parent Document Retrieval

What It Is:

A retrieval method that structures documents hierarchically — into parent and child nodes — so large texts can be efficiently searched and summarized for LLM or RAG systems.

Key Idea:

Instead of retrieving every small chunk independently, the system first retrieves **parent nodes** (larger sections) to identify the most relevant context, then expands only those by including their c**hild nodes** (smaller text segments or embeddings).

How It Works:

1. Divide documents into hierarchical units: book → sections → paragraphs.
2. Embed both parent and child nodes separately.
3. Retrieve top parent nodes relevant to the query.
4. Expand those parents by adding associated child nodes to the context.
5. Construct a focused, context-rich prompt from the expanded results.

The Problem

- Vanilla RAG = **search** + **summarize**
- Great for static Q&A, but…
  - No reasoning across multiple steps
  - No integration with APIs or actions
  - Limited autonomy

From RAG to Agentic RAG

- **Agentic RAG = RAG + Planning + Tool Use**
  - Adds a **reasoning loop**: retrieve → reason → act → reflect
  - The model **decides what to retrieve and what to do next**
  - Enables **dynamic task completion**, not just text generation

Agentic RAG Workflow

1. **User Query** → goal or question
2. **Planner LLM** decides steps
3. **Retriever LLM / Vector DB** fetches relevant context
4. **Tool calls** (APIs, functions) execute tasks
5. **Reasoner LLM** evaluates results and iterates
6. **Final answer/action output**

OpenAI Tools – Function Definition

```python
TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "find_coins_by_imperor",
            "description": "Find Roman coins associated with a specific emperor or authority. Use this when the user asks about coins minted under a particular emperor's reign or showing a specific emperor's portrait.",
            "parameters": {
                "type": "object",
                "properties": {
                    "name": {
                        "type": "string",
                        "description": "The name of the emperor or authority (e.g., 'Constantine I', 'Caracalla', 'Augustus')",
                    },
                    "limit": {
                        "type": "integer",
                        "description": "Maximum number of coins to return. Default is 10.",
                        "default": 10,
                    },
                },
                "required": ["name"],
            },
        },
    }
]
```

OpenAI Tools - Usage

```python
def chat(self, query: str, history: list[LLMMessage] | None = None) -> str:
    # Step 1: Get tool calls from LLM
    response = self._client.chat.completions.create(
        model=settings.chat_model,
        messages=[
            {"role": "system", "content": TOOL_PICKER_SYSTEM_PROMPT},
            {"role": "user", "content": query},
        ],
        tools=TOOLS,
        tool_choice="auto",
        temperature=settings.temperature,
        top_p=settings.top_p,
    )

    message = response.choices[0].message

    # Step 2: Execute tools and collect results
    if message.tool_calls:
        logger.info(f"LLM called {len(message.tool_calls)} tool(s):\n")

        all_coins = []
        tool_info = []

        for tool_call in message.tool_calls:
            function_name = tool_call.function.name
            function_args = json.loads(tool_call.function.arguments)

            logger.info(f"Tool: {function_name}")
            logger.info(f"Arguments: {function_args}")

            function_to_call = AVAILABLE_FUNCTIONS.get(function_name)
            if function_to_call:
                tool_response = function_to_call(**function_args)
                logger.info(f"Results: Found {len(tool_response)} coins")
```

What are MCP Servers?

- **Model Context Protocol (MCP) servers** are standardized interfaces that allow AI assistants like Claude to connect to external data sources and tools. They act as bridges between AI models and various services, enabling dynamic access to information and capabilities beyond the model's training data.
- **Key Features**
  - **Standardized Protocol:** Consistent interface for connecting AI models to external resources
  - **Extensible Architecture:** Support for various data sources (databases, APIs, file systems, etc.)
  - **Real-time Access:** Enables AI to retrieve current information and interact with live systems
  - **Tool Integration:** Allows AI to execute functions and operations through connected services
  - **Security & Control:** Provides controlled access to sensitive data and operations

How the Flow Works

- **LLM → MCP Client**: “**describe_roman_coins** Coins from the late Roman Empire”
- **MCP Client → Server:** Sends a standardized JSON RPC request.
- **Server**: Executes your code / API and returns results.
- **Client → LLM:** Feeds results back into the conversation context.

What Is FastMCP?

- **FastMCP** = reference implementation of the **Model Context Protocol (MCP)** built with **FastAPI + async I/O.**
- Designed for **fast**, **scalable**, and **developer-friendly** deployment of MCP tool servers.
- **FastMCP** Server handles JSON-RPC, schema validation, and routing.
- **Your Functions** define the actual logic (DB queries, RAG retrieval, etc.).
- Fully async → high concurrency and low latency.

Expose Agentic RAG as MCP Server

```python
from fastmcp import FastMCP
from core.ocre_agent import AgenticRAGService


mcp = FastMCP("roman-numismatics-assistant")


@mcp.tool
def describe_roman_coins(query: str) -> str:
    agent = AgenticRAGService()
    return agent.chat(query)
```

Use in Claude Desktop

```
{
        "mcpServers": {
                "roman-numismatics-assistant": {
                        "command": "uv",
                        "args": [
                                "--directory",
                                "/Users/user/repo/neo4j-genai-ocre",
                                "run",
                                "mcp_server.py"
                        ]
                }
        }
}
```

Thank you!

Contact details

- **Website:** https://verbitskiy.co
- **GitHub:** https://github.com/ilich
- **X:** @ilich_x86